Datapath and Controller Design

1 Introduction

Digital systems are often organized into two cooperating parts. The *datapath* stores and transforms values using registers, arithmetic and logic units, multiplexers, counters, and buses. The *controller* is a finite-state machine (FSM) that sequences those operations by issuing control signals. Status signals, such as a zero or comparison result, flow back from the datapath to help the controller choose its next state.

The sketch below gives a small register-transfer example. The datapath performs the additions and stores their results; the controller enables the relevant registers at the right time.

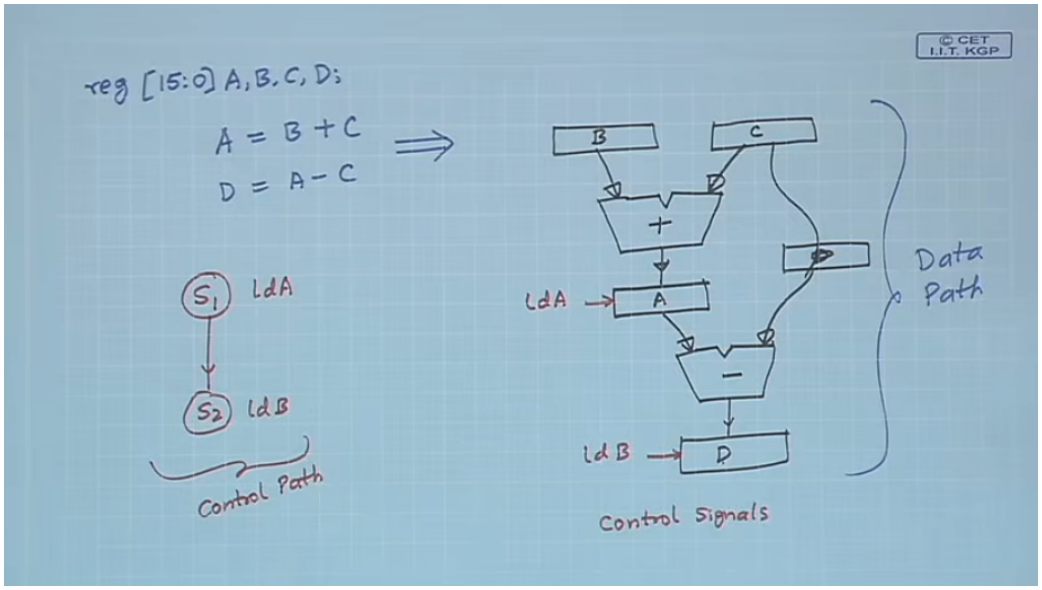


Figure 1: Register-transfer and control-path sketch from the source notes.

The examples that follow use Verilog. The code is presented as editable source listings rather than screenshots. Signal names and structure follow the source notes where practical; incomplete screenshot details are normalized into complete examples.

2 Multiplication by Repeated Addition

For unsigned inputs A and B, repeated addition computes $A \times B$ by initializing the product P to zero, adding A to P, and decrementing a counter initialized to B. The process stops when the counter reaches zero. The datapath therefore needs an operand register, an accumulator, and a counter; the controller sequences their loads, additions, and decrements.

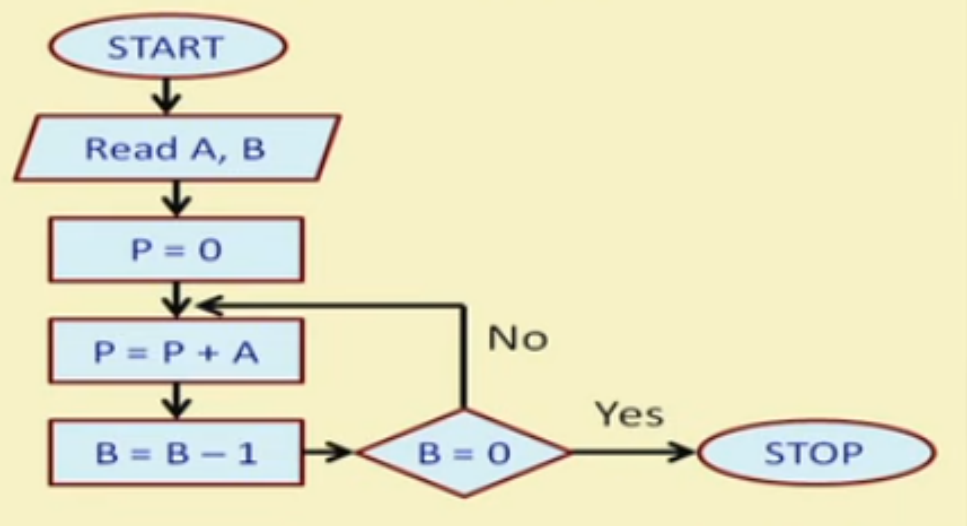


Figure 2: Algorithm flow for multiplication by repeated addition.

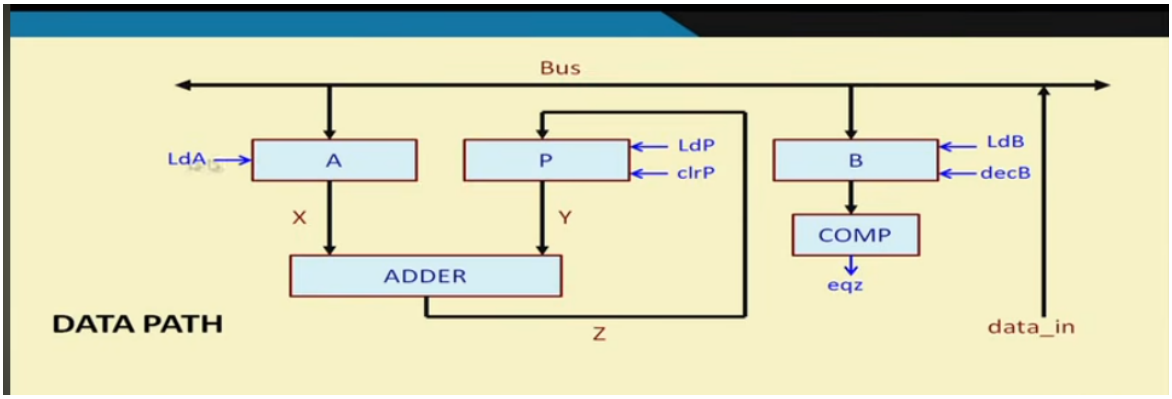


Figure 3: Repeated-addition datapath.

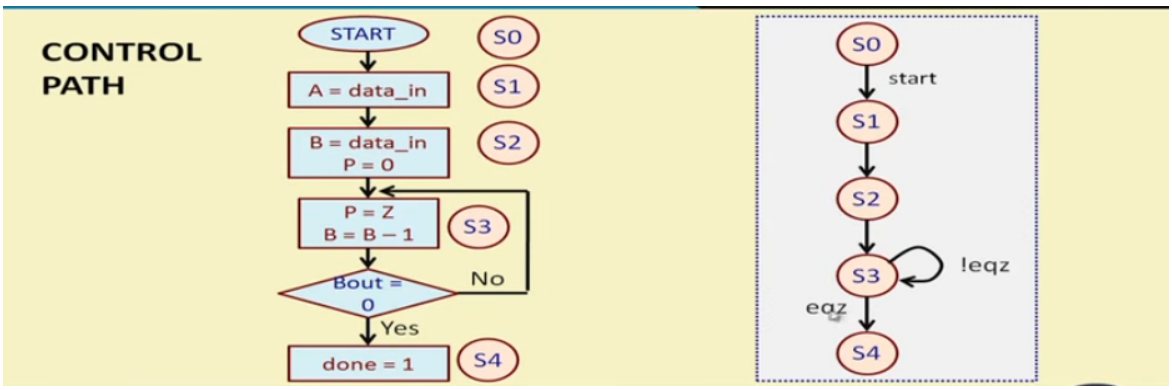


Figure 4: Control-flow and state diagram for the repeated-addition example.

The source below puts the datapath, controller, and testbench in an editor-style listing. State changes occur at clock edges, while the combinational blocks determine the next state and control outputs.

Listing 1: Repeated-addition multiplier and testbench

```
module mul_datapath (
    input clk, load_operand, load_count, clear_product,
    input accumulate, decrement_count,
    input [15:0] multiplicand_in, multiplier_in,
    output reg [15:0] product, count,
    output count_zero
);
    reg [15:0] operand;

    always @(posedge clk) begin
        if (load_operand) operand <= multiplicand_in;
        if (clear_product) product <= 16'b0;
        else if (accumulate) product <= product + operand;
        if (load_count) count <= multiplier_in;
        else if (decrement_count) count <= count - 1'b1;
    end

    assign count_zero = (count == 16'b0);
endmodule

module mul_controller (
    input clk, reset, start, count_zero,
    output reg load_operand, load_count, clear_product,
    output reg accumulate, decrement_count,
    output done
);
    localparam IDLE = 3'd0, LOAD_A = 3'd1, LOAD_B = 3'd2,
               CHECK = 3'd3, ADD = 3'd4, FINISH = 3'd5;
    reg [2:0] state, next_state;

    always @(posedge clk)
        if (reset) state <= IDLE;
        else state <= next_state;

    always @(*) begin
        next_state = state;
        case (state)
            IDLE: if (start) next_state = LOAD_A;
            LOAD_A: next_state = LOAD_B;
            LOAD_B: next_state = CHECK;
            CHECK: next_state = count_zero ? FINISH : ADD;
            ADD: next_state = CHECK;
            FINISH: next_state = IDLE;
            default: next_state = IDLE;
        endcase
    end

    always @(*) begin
        load_operand = 0;
        load_count = 0;
        clear_product = 0;
        accumulate = 0;
        decrement_count = 0;
        case (state)
            LOAD_A: load_operand = 1;
            LOAD_B: begin load_count = 1; clear_product = 1; end
            ADD: begin accumulate = 1; decrement_count = 1; end
            default: begin end
        endcase
    end
```

```
61
62      assign done = (state == FINISH);
63  endmodule
64
65  module repeated_addition_multiplier (
66      input clk, reset, start,
67      input [15:0] multiplicand, multiplier,
68      output [15:0] product,
69      output done
70  );
71      wire load_operand, load_count, clear_product;
72      wire accumulate, decrement_count, count_zero;
73      wire [15:0] count;
74
75      mul_datapath datapath (
76          .clk(clk), .load_operand(load_operand), .load_count(load_count),
77          .clear_product(clear_product), .accumulate(accumulate),
78          .decrement_count(decrement_count), .multiplicand_in(multiplicand),
79          .multiplier_in(multiplier), .product(product), .count(count),
80          .count_zero(count_zero)
81      );
82      mul_controller controller (
83          .clk(clk), .reset(reset), .start(start), .count_zero(count_zero),
84          .load_operand(load_operand), .load_count(load_count),
85          .clear_product(clear_product), .accumulate(accumulate),
86          .decrement_count(decrement_count), .done(done)
87      );
88  endmodule
89
90  module repeated_addition_test;
91      reg clk = 0, reset = 1, start = 0;
92      wire [15:0] product;
93      wire done;
94      always #5 clk = ~clk;
95
96      repeated_addition_multiplier dut (
97          .clk(clk), .reset(reset), .start(start),
98          .multiplicand(16'd17), .multiplier(16'd5),
99          .product(product), .done(done)
100     );
101
102     initial begin
103         #12 reset = 0;
104         #8 start = 1;
105         #10 start = 0;
106         wait (done);
107         #1;
108         if (product !== 16'd85) $fatal(1, "multiplier test failed");
109         $display("17 x 5 = %0d", product);
110         $finish;
111     end
112 endmodule
```

3 Separating State Updates from Control Logic

A useful RTL style updates the state register in a clocked block, while calculating the next state and outputs in separate combinational blocks. State changes then occur only on a clock edge, and the com-

binational logic describes the next state and control signals. Avoid simulation delays such as `#2` in synthesizable controller logic.

4 Greatest Common Divisor by Repeated Subtraction

For positive integers A and B, the subtraction method for the greatest common divisor (GCD) repeatedly replaces the larger value with the difference between the two. When both values are equal, that common value is the GCD. A comparator supplies status signals; multiplexers select the subtractor operands and the register to update.

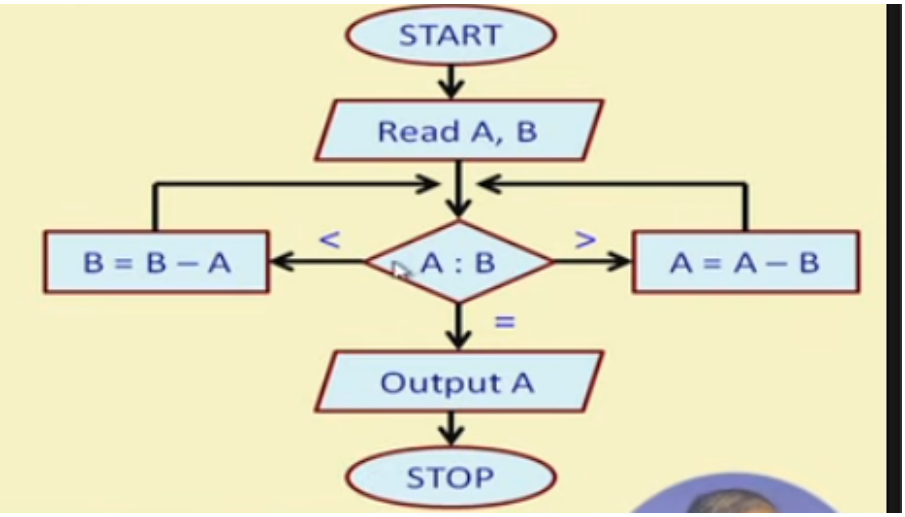


Figure 5: Repeated-subtraction steps for the GCD.

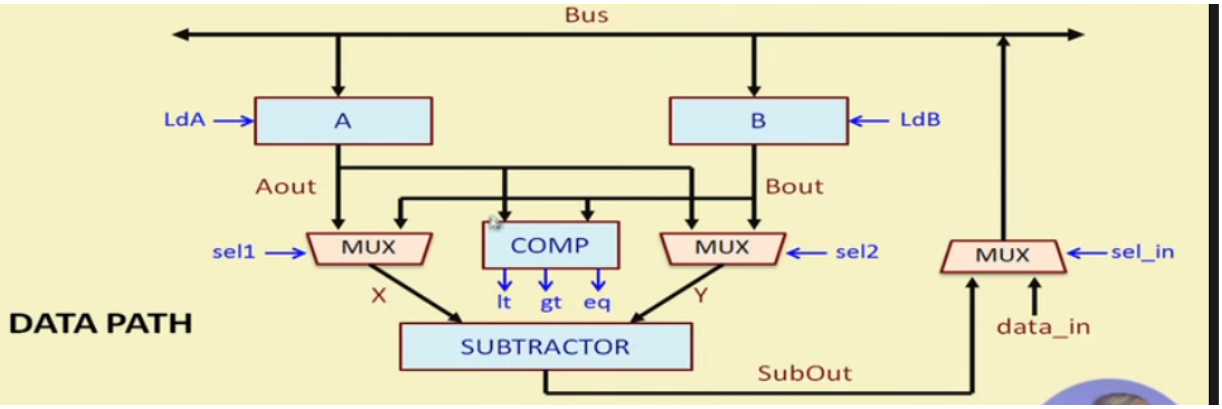


Figure 6: GCD datapath with registers, comparator, multiplexers, and subtractor.

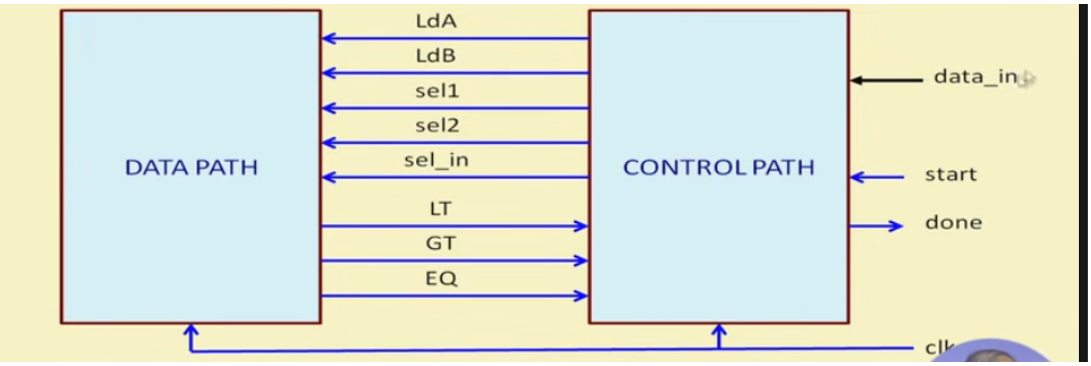


Figure 7: Control and datapath interface for the GCD example.

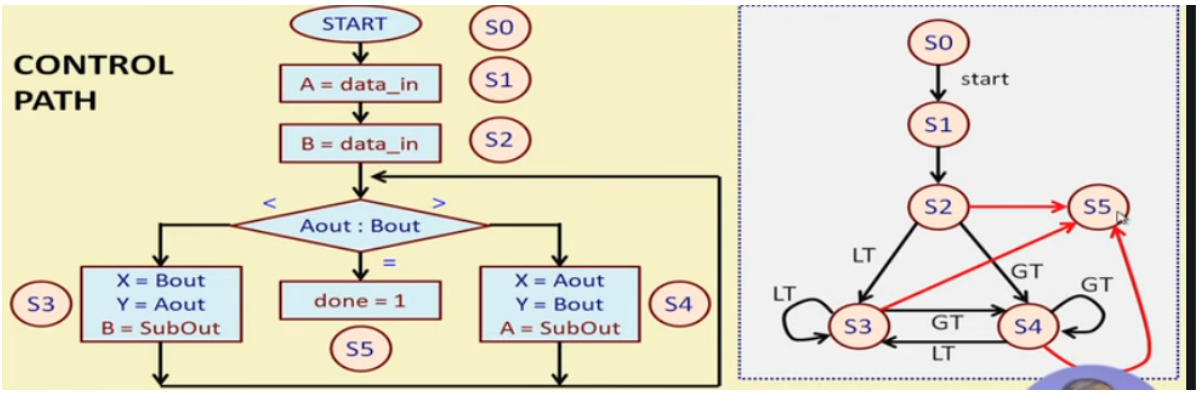


Figure 8: GCD controller state diagram.

The example uses 143 and 78, whose GCD is 13. The code separates datapath modules from the controller and demonstrates the clocked-state/combinational-output style.

Listing 2: GCD datapath, controller, and testbench

```
module PIPO (
    input clk, load,
    input [15:0] data_in,
    output reg [15:0] data_out
);
    always @(posedge clk) if (load) data_out <= data_in;
endmodule

module SUB (input [15:0] in1, in2, output [15:0] out);
    assign out = in1 - in2;
endmodule

module COMPARE (
    input [15:0] data1, data2,
    output lt, gt, eq
);
    assign lt = data1 < data2;
    assign gt = data1 > data2;
    assign eq = data1 == data2;
endmodule

module MUX (
    input [15:0] in0, in1,
```

```
    input select,
    output [15:0] out
);
    assign out = select ? in1 : in0;
endmodule

module gcd_datapath (
    input clk, load_a, load_b, update_a, update_b, choose_b,
    input [15:0] data_a, data_b,
    output [15:0] a_out, b_out,
    output lt, gt, eq
);
    wire [15:0] x, y, difference, a_next, b_next;
    wire load_a_reg = load_a | update_a;
    wire load_b_reg = load_b | update_b;

    MUX select_x (a_out, b_out, choose_b, x);
    MUX select_y (b_out, a_out, choose_b, y);
    SUB subtractor (x, y, difference);
    MUX input_a (difference, data_a, load_a, a_next);
    MUX input_b (difference, data_b, load_b, b_next);
    PIPO register_a (clk, load_a_reg, a_next, a_out);
    PIPO register_b (clk, load_b_reg, b_next, b_out);
    COMPARE compare_values (a_out, b_out, lt, gt, eq);
endmodule

module gcd_controller (
    input clk, reset, start, lt, gt, eq,
    output reg load_a, load_b, update_a, update_b, choose_b,
    output done
);
    localparam IDLE = 3'd0, LOAD_A = 3'd1, LOAD_B = 3'd2,
               COMPARE_VALUES = 3'd3, UPDATE_A = 3'd4,
               UPDATE_B = 3'd5, FINISH = 3'd6;
    reg [2:0] state, next_state;

    always @(posedge clk)
        if (reset) state <= IDLE;
        else state <= next_state;

    always @(*) begin
        next_state = state;
        case (state)
            IDLE: if (start) next_state = LOAD_A;
            LOAD_A: next_state = LOAD_B;
            LOAD_B: next_state = COMPARE_VALUES;
            COMPARE_VALUES: begin
                if (eq) next_state = FINISH;
                else if (lt) next_state = UPDATE_B;
                else if (gt) next_state = UPDATE_A;
            end
            UPDATE_A, UPDATE_B: next_state = COMPARE_VALUES;
            FINISH: next_state = IDLE;
            default: next_state = IDLE;
        endcase
    end

    always @(*) begin
        load_a = 0; load_b = 0;
        update_a = 0; update_b = 0; choose_b = 0;
        case (state)
```

```
            LOAD_A: load_a = 1;
            LOAD_B: load_b = 1;
            UPDATE_A: update_a = 1;
            UPDATE_B: begin update_b = 1; choose_b = 1; end
            default: begin end
        endcase
    end

    assign done = (state == FINISH);
endmodule

module gcd_repeated_subtraction (
    input clk, reset, start,
    input [15:0] data_a, data_b,
    output [15:0] gcd,
    output done
);
    wire load_a, load_b, update_a, update_b, choose_b;
    wire [15:0] b_out;
    wire lt, gt, eq;

    gcd_datapath datapath (
        clk, load_a, load_b, update_a, update_b, choose_b,
        data_a, data_b, gcd, b_out, lt, gt, eq
    );
    gcd_controller controller (
        clk, reset, start, lt, gt, eq,
        load_a, load_b, update_a, update_b, choose_b, done
    );
endmodule

module gcd_test;
    reg clk = 0, reset = 1, start = 0;
    wire [15:0] gcd;
    wire done;
    always #5 clk = ~clk;

    gcd_repeated_subtraction dut (
        .clk(clk), .reset(reset), .start(start),
        .data_a(16'd143), .data_b(16'd78), .gcd(gcd), .done(done)
    );

    initial begin
        #12 reset = 0;
        #8 start = 1;
        #10 start = 0;
        wait (done);
        #1;
        if (gcd !== 16'd13) $fatal(1, "GCD test failed");
        $display("gcd(143, 78) = %0d", gcd);
        $finish;
    end
endmodule
```

5 Booth's Multiplication

Ordinary shift-and-add multiplication can require one addition for every multiplier bit. Booth's algorithm examines (Q_0, Q_{-1}), with Q_{-1} initialized to zero. Pair 00 or 11 means shift only; 01 means add the

multiplicand; and 10 means subtract it. After the selected operation, the partial product and multiplier shift arithmetically to the right. Runs of equal bits can therefore reduce the number of additions or subtractions.

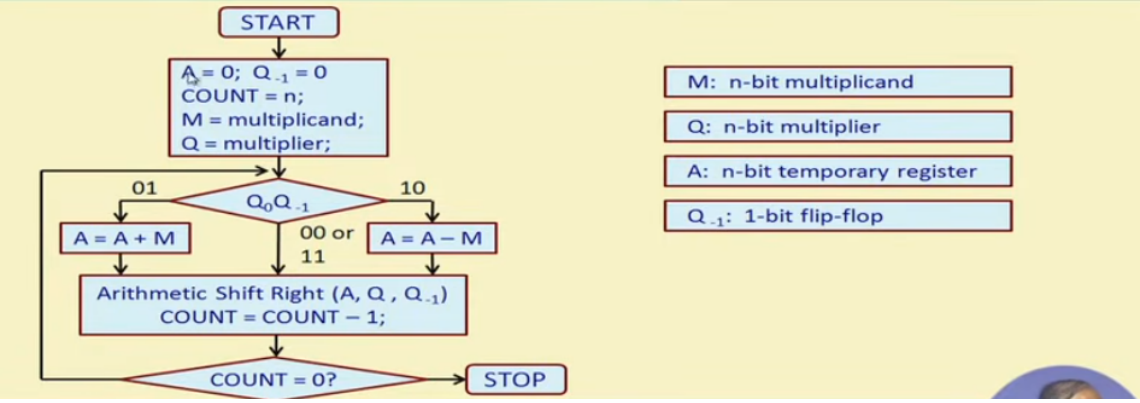


Figure 9: Booth's algorithm flow chart and register definitions.

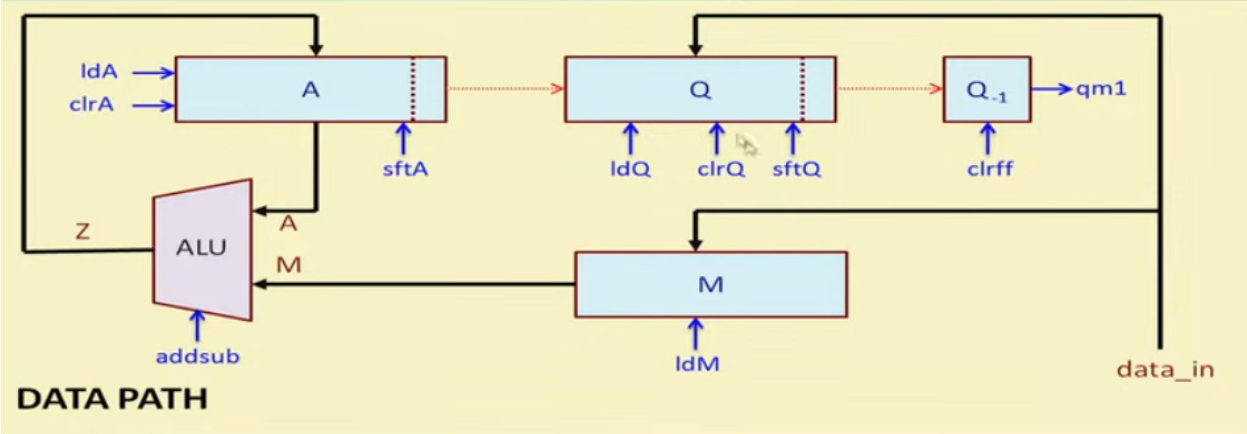


Figure 10: Booth multiplier datapath.

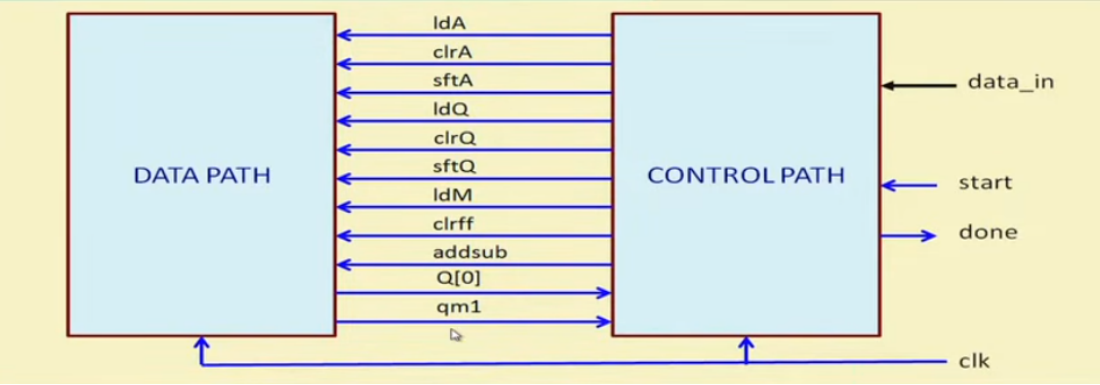


Figure 11: Booth datapath/controller interface.

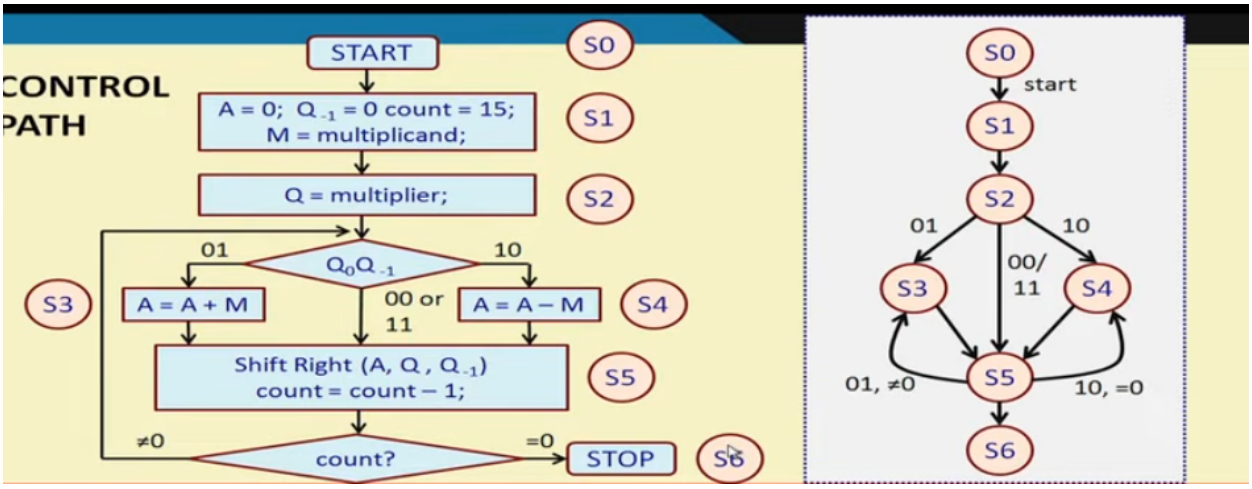


Figure 12: Booth control flow and FSM.

This signed 16-bit example shows the iteration controller and arithmetic registers directly in Verilog. The small helper modules below mirror the register, ALU, and counter blocks in the datapath figure.

Listing 3: Booth multiplier controller, datapath, and testbench

```
module shiftreg #(
    parameter WIDTH = 16
) (
    input clk, load, clear, shift, serial_in,
    input [WIDTH-1:0] data_in,
    output reg [WIDTH-1:0] data_out
);
    always @(posedge clk) begin
        if (clear) data_out <= {WIDTH{1'b0}};
        else if (load) data_out <= data_in;
        else if (shift) data_out <= {serial_in, data_out[WIDTH-1:1]};
    end
endmodule

module PIPO #(
    parameter WIDTH = 16
) (
    input clk, load,
    input [WIDTH-1:0] data_in,
    output reg [WIDTH-1:0] data_out
);
    always @(posedge clk) if (load) data_out <= data_in;
endmodule

module ALU #(
    parameter WIDTH = 17
) (
    input [WIDTH-1:0] in1, in2,
    input addsub,
    output [WIDTH-1:0] out
);
    assign out = addsub ? in1 + in2 : in1 - in2;
endmodule

module booth_multiplier (
    input clk, reset, start,
```

```
    input signed [15:0] multiplicand, multiplier,
    output signed [31:0] product,
    output done
);
    localparam IDLE = 3'd0, INITIALIZE = 3'd1,
               CHECK_PAIR = 3'd2, ADD = 3'd3,
               SUBTRACT = 3'd4, SHIFT = 3'd5, FINISH = 3'd6;
    reg [2:0] state, next_state;
    reg signed [16:0] accumulator, multiplicand_reg;
    reg signed [15:0] quotient;
    reg previous_bit;
    reg [4:0] count;
    reg signed [16:0] adjusted;
    wire signed [33:0] shifted_value;
    wire signed [16:0] signed_multiplicand =
        {multiplicand[15], multiplicand};

    always @(*) begin
        case ({quotient[0], previous_bit})
            2'b01: adjusted = accumulator + multiplicand_reg;
            2'b10: adjusted = accumulator - multiplicand_reg;
            default: adjusted = accumulator;
        endcase
    end

    assign shifted_value = $signed({adjusted, quotient, previous_bit}) >>> 1;
    assign product = {accumulator[15:0], quotient};
    assign done = (state == FINISH);

    always @(posedge clk) begin
        if (reset) begin
            state <= IDLE;
            accumulator <= 0;
            quotient <= 0;
            previous_bit <= 0;
            multiplicand_reg <= 0;
            count <= 0;
        end else begin
            state <= next_state;
            case (state)
                INITIALIZE: begin
                    accumulator <= 0;
                    previous_bit <= 0;
                    multiplicand_reg <= signed_multiplicand;
                    quotient <= multiplier;
                    count <= 5'd16;
                end
                SHIFT: begin
                    {accumulator, quotient, previous_bit} <= shifted_value;
                    count <= count - 1'b1;
                end
                default: begin end
            endcase
        end
    end

    always @(*) begin
        next_state = state;
        case (state)
            IDLE: if (start) next_state = INITIALIZE;
            INITIALIZE: next_state = CHECK_PAIR;
```

```
            CHECK_PAIR: begin
                if (count == 0) next_state = FINISH;
                else if ({quotient[0], previous_bit} == 2'b01)
                    next_state = ADD;
                else if ({quotient[0], previous_bit} == 2'b10)
                    next_state = SUBTRACT;
                else next_state = SHIFT;
            end
            ADD, SUBTRACT: next_state = SHIFT;
            SHIFT: next_state = CHECK_PAIR;
            FINISH: if (!start) next_state = IDLE;
            default: next_state = IDLE;
        endcase
    end
endmodule

module booth_test;
    reg clk = 0, reset = 1, start = 0;
    wire signed [31:0] product;
    wire done;
    always #5 clk = ~clk;

    booth_multiplier dut (
        .clk(clk), .reset(reset), .start(start),
        .multiplicand(-16'sd13), .multiplier(16'sd7),
        .product(product), .done(done)
    );

    initial begin
        #12 reset = 0;
        #8 start = 1;
        #10 start = 0;
        wait (done);
        #1;
        if (product !== -32'sd91) $fatal(1, "Booth test failed");
        $display("-13 x 7 = %0d", product);
        $finish;
    end
endmodule
```

6 Testbench Notes

A testbench should drive inputs and start signals at deliberate times, observe outputs after clocked operations have settled, and finish after the expected result is available. Check that tests cover zero, equal operands, and both less-than and greater-than comparisons.